

INSTITUTO TECNOLÓGICO SUPERIOR DEL ORIENTE DEL ESTADO DE HIDALGO

Ingeniería en Sistemas Computacionales

Docente: Efren Rolando Romero Leon

Materia: Métodos Numéricos

Problemario

Alumnos:

Briana Alexa Del Valle Guzmán **24030220**

Alan Aguilar De Lucio **24030053**

Raymundo Ortega Santelis **24030450**

Periodo: Enero – Julio

INTERPOLACIÓN LINEAL

Interpolación lineal

La cuota o término de error es:

$$E(x) = \frac{f''(\xi)}{2!}(x - x_0)(x - x_1)$$

Donde:

- $E(x)$ = error de interpolación
- $f''(\xi)$ = segunda derivada de la función real
- ξ = algún punto dentro del intervalo
- x_0, x_1 = puntos conocidos

Interpretación

El error depende de:

- Qué tan curva sea la función
- La distancia entre puntos

Mientras más cercanos estén los puntos, menor será el error.

Ejemplo 1

Planteamiento

La temperatura de un horno es de 120 °C a los 20 minutos y de 60 °C a los 40 minutos. Calcular la temperatura aproximada a los 30 minutos usando interpolación lineal.

Fórmula y datos

Datos:

$$x_0 = 20 \quad y_0 = 120$$

$$x_1 = 40 \quad y_1 = 60$$

$$x = 30$$

Fórmula:

$$y = y_0 + \frac{(x - x_0)(y_1 - y_0)}{x_1 - x_0}$$

Resolución

Sustituyendo:

$$y = 120 + \frac{(30 - 20)(60 - 120)}{40 - 20}$$

$$y = 120 + \frac{(10)(-60)}{20}$$

$$Y = 120 - 30$$

$$y = 90$$

Interpretación de resultados

La temperatura aproximada del horno a los 30 minutos es: 90

Pseudocódigo

Inicio

Definir $x_0=20$, $y_0=120$

Definir $x_1=40$, $y_1=60$

Definir $x=30$

$y = y_0 + ((x-x_0)*(y_1-y_0)) / (x_1-x_0)$

Mostrar y

Fin

Código en python

```
x0 = 20

y0 = 120


x1 = 40

y1 = 60
```

```
x = 30

y = y0 + ((x - x0) * (y1 - y0)) / (x1 - x0)

print("Resultado:", y)
```

Resultado obtenido

```
Resultado: 90.0
```

Ejemplo 2

Planteamiento

Un automóvil recorre 100 km en 2 horas y 180 km en 4 horas. Calcular la distancia aproximada recorrida en 3 horas.

Fórmula y datos

Datos:

$$x_0 = 2 \quad y_0 = 100$$

$$x_1 = 4 \quad y_1 = 180$$

$$x = 3$$

Fórmula:

$$y = y_0 + \frac{(x - x_0)(y_1 - y_0)}{x_1 - x_0}$$

Resolución

Sustituyendo:

$$y = 100 + \frac{(3 - 2)(180 - 100)}{4 - 2}$$

$$y = 100 + \frac{(1)(80)}{2}$$

$$Y = 100 + 40$$

$$y = 140$$

Interpretación de resultados

La distancia aproximada recorrida en 3 horas es: 140 km

Pseudocódigo

Inicio

Definir $x_0=2$, $y_0=100$

Definir $x_1=4$, $y_1=180$

Definir $x=3$

$y = y_0 + ((x-x_0)*(y_1-y_0)) / (x_1-x_0)$

Mostrar y

Fin

Código en python

```
x0 = 2

y0 = 100


x1 = 4

y1 = 180


x = 3


y = y0 + ((x - x0) * (y1 - y0)) / (x1 - x0)
```

```
print("Resultado:", y)
```

Resultado obtenido

```
Resultado: 140.0
```

Ejemplo 3

Planteamiento

Una sustancia pesa 50 g a los 10 segundos y 90 g a los 30 segundos. Calcular el peso aproximado a los 20 segundos.

Fórmula y datos

Datos:

$$x_0 = 10 \quad y_0 = 50$$

$$x_1 = 30 \quad y_1 = 90$$

$$x = 20$$

Fórmula:

$$y = y_0 + \frac{(x - x_0)(y_1 - y_0)}{x_1 - x_0}$$

Resolución

Sustituyendo:

$$y = 50 + \frac{(20 - 10)(90 - 50)}{30 - 10}$$

$$y = 50 + \frac{(10)(40)}{20}$$

$$Y = 50 + 20$$

$$y = 70$$

Interpretación de resultados

El peso aproximado a los 20 segundos es: 70 g

Pseudocódigo

Inicio

Definir $x_0=10$, $y_0=50$

Definir $x_1=30$, $y_1=90$

Definir $x=20$

$y = y_0 + ((x-x_0)*(y_1-y_0)) / (x_1-x_0)$

Mostrar y

Fin

Código en python

```
x0 = 10

y0 = 50


x1 = 30

y1 = 90


x = 20


y = y0 + ((x - x0)*(y1 - y0))/(x1 - x0)


print("Resultado:", y)
```

Resultado obtenido **Resultado: 70.0**

Ejemplo 4

Planteamiento

El costo de producción es de \$500 para 5 piezas y de \$900 para 9 piezas.
Calcular el costo aproximado para 7 piezas.

Fórmula y datos

Datos:

Fórmula:

$$\begin{aligned}x_0 &= 5 & y_0 &= 500 \\x_1 &= 9 & y_1 &= 900 \\x &= 7\end{aligned}$$
$$y = y_0 + \frac{(x - x_0)(y_1 - y_0)}{x_1 - x_0}$$

Resolución

Sustituyendo:

$$y = 500 + \frac{(7 - 5)(900 - 500)}{9 - 5}$$

$$y = 500 + \frac{(2)(400)}{4}$$

$$Y = 500 + 200$$

$$y = 700$$

Interpretación de resultados

El costo aproximado para 7 piezas es: \$700

Pseudocódigo

Inicio

Definir $x_0=5$, $y_0=500$

Definir $x_1=9$, $y_1=900$

Definir $x=7$

$y = y_0 + ((x-x_0)*(y_1-y_0)) / (x_1-x_0)$

Mostrar y

Fin

Código en python

```
x0 = 5
y0 = 500

x1 = 9
y1 = 900

x = 7

y = y0 + ((x - x0)*(y1 - y0))/(x1 - x0)

print("Resultado:", y)
```

Resultado obtenido

Resultado: 700.0

INTERPOLACIÓN CUADRÁTICA

Interpolación cuadrática

La cuota o término de error es:

$$E(x) = \frac{f'''(\xi)}{3!}(x - x_0)(x - x_1)(x - x_2)$$

Donde:

- $f'''(\xi)$ = tercera derivada de la función
- x_0, x_1, x_2 = puntos usados

Interpretación

La interpolación cuadrática normalmente tiene menor error que la lineal porque usa 3 puntos y se adapta mejor a curvas.

Ejemplo 1

Planteamiento

Un laboratorio registra que una bacteria tiene un crecimiento de:

- 2 mil bacterias en 1 hora
- 5 mil bacterias en 2 horas
- 17 mil bacterias en 4 horas

Calcular la cantidad aproximada de bacterias a las 3 horas utilizando interpolación cuadrática de Lagrange

Fórmula general

$$y = y_0L_0(x) + y_1L_1(x) + y_2L_2(x)$$

Datos

$$x_0 = 1, \quad y_0 = 2$$

$$x_1 = 2, \quad y_1 = 5$$

$$x_2 = 4, \quad y_2 = 17$$

$$x = 3$$

Resolución

Paso 1: Calcular $L_0(x)$

$$\begin{aligned}L_0(3) &= \frac{(3-2)(3-4)}{(1-2)(1-4)} \\&= \frac{(1)(-1)}{(-1)(-3)} \\&= \frac{-1}{3} \\L_0(3) &= -0.333\end{aligned}$$

Paso 2: Calcular $L_1(x)$

$$\begin{aligned}L_1(3) &= \frac{(3-1)(3-4)}{(2-1)(2-4)} \\&= \frac{(2)(-1)}{(1)(-2)} \\&= \frac{-2}{-2} \\L_1(3) &= 1\end{aligned}$$

Paso 3: Calcular $L_2(x)$

$$\begin{aligned}L_2(3) &= \frac{(3-1)(3-2)}{(4-1)(4-2)} \\&= \frac{(2)(1)}{(3)(2)} \\&= \frac{2}{6} \\L_2(3) &= 0.333\end{aligned}$$

Paso 4: Sustituir en la fórmula

$$y = 2(-0.333) + 5(1) + 17(0.333)$$

$$y = -0.666 + 5 + 5.661$$

$$y = 9.995$$

$$y \approx 10$$

Interpretación de resultados

La cantidad aproximada de bacterias a las 3 horas es: **10 mil bacterias**

Pseudocódigo

Inicio

Definir $x_0=1$, $y_0=2$

Definir $x_1=2$, $y_1=5$

Definir $x_2=4$, $y_2=17$

Definir $x=3$

$L_0 = ((x-x_1)*(x-x_2))/((x_0-x_1)*(x_0-x_2))$

$L_1 = ((x-x_0)*(x-x_2))/((x_1-x_0)*(x_1-x_2))$

$L_2 = ((x-x_0)*(x-x_1))/((x_2-x_0)*(x_2-x_1))$

$y = (y_0*L_0) + (y_1*L_1) + (y_2*L_2)$

Mostrar y

Fin

Código en python

```
x0, y0 = 1, 2
```

```
x1, y1 = 2, 5
```

```
x2, y2 = 4, 17

x = 3

L0 = ((x-x1)*(x-x2)) / ((x0-x1)*(x0-x2))
L1 = ((x-x0)*(x-x2)) / ((x1-x0)*(x1-x2))
L2 = ((x-x0)*(x-x1)) / ((x2-x0)*(x2-x1))

y = y0*L0 + y1*L1 + y2*L2

print("Resultado:", y)
```

Resultado obtenido

Resultado: 10.0

Ejemplo 2

Planteamiento

Una empresa registra las ganancias de un producto:

- \$1,000 cuando se venden 0 unidades
- \$4,000 cuando se venden 2 unidades
- \$10,000 cuando se venden 5 unidades

Calcular la ganancia aproximada cuando se venden 3 unidades usando interpolación cuadrática.

Fórmula y datos

$$y = y_0 L_0(x) + y_1 L_1(x) + y_2 L_2(x)$$

Datos

$$x_0 = 0, \quad y_0 = 1$$

$$x_1 = 2, \quad y_1 = 4$$

$$x_2 = 5, \quad y_2 = 10$$

$$x = 3$$

Resolución

Paso 1: Calcular $L_0(x)$

$$L_0(3) = \frac{(3 - 2)(3 - 5)}{(0 - 2)(0 - 5)}$$

$$= \frac{(1)(-2)}{(-2)(-5)}$$

$$= \frac{-2}{10}$$

$$L_0(3) = -0.2$$

Paso 2: Calcular $L_1(x)$

$$\begin{aligned} L_1(3) &= \frac{(3-0)(3-5)}{(2-0)(2-5)} \\ &= \frac{(3)(-2)}{(2)(-3)} \\ &= \frac{-6}{-6} \\ L_1(3) &= 1 \end{aligned}$$

Paso 3: Calcular $L_2(x)$

$$\begin{aligned} L_2(3) &= \frac{(3-0)(3-2)}{(5-0)(5-2)} \\ &= \frac{(3)(1)}{(5)(3)} \\ &= \frac{3}{15} \\ L_2(3) &= 0.2 \end{aligned}$$

Paso 4: Sustituir en la fórmula

$$\begin{aligned} y &= 1(-0.2) + 4(1) + 10(0.2) \\ y &= -0.2 + 4 + 2 \\ y &= 5.8 \end{aligned}$$

Interpretación de resultados

La ganancia aproximada al vender 3 unidades es: **\$5,800**

Pseudocódigo

Inicio

Definir $x_0=0$, $y_0=1$

Definir $x_1=2$, $y_1=4$

Definir $x_2=5$, $y_2=10$

Definir $x=3$

$L_0 = ((x-x_1)*(x-x_2))/((x_0-x_1)*(x_0-x_2))$

$L_1 = ((x-x_0)*(x-x_2))/((x_1-x_0)*(x_1-x_2))$

$L_2 = ((x-x_0)*(x-x_1))/((x_2-x_0)*(x_2-x_1))$

$y = (y_0*L_0) + (y_1*L_1) + (y_2*L_2)$

Mostrar y

Fin

Código en python

```
x0, y0 = 0, 1

x1, y1 = 2, 4

x2, y2 = 5, 10

x = 3

L0 = ((x-x1)*(x-x2))/((x0-x1)*(x0-x2))

L1 = ((x-x0)*(x-x2))/((x1-x0)*(x1-x2))

L2 = ((x-x0)*(x-x1))/((x2-x0)*(x2-x1))

y = y0*L0 + y1*L1 + y2*L2
```



```
print("Resultado:", y)
```

Resultado obtenido

```
Resultado: 5.8
```

Ejemplo 3

Planteamiento

Un automóvil registra las siguientes distancias recorridas:

- 1 km en 1 minuto
- 9 km en 3 minutos
- 25 km en 5 minutos

Calcular la distancia aproximada recorrida a los 4 minutos usando interpolación cuadrática.

Fórmula y datos

$$y = y_0L_0(x) + y_1L_1(x) + y_2L_2(x)$$

Datos

$$x_0 = 1, \quad y_0 = 1$$

$$x_1 = 3, \quad y_1 = 9$$

$$x_2 = 5, \quad y_2 = 25$$

$$x = 4$$

Resolución

Paso 1: Calcular $L_0(x)$

$$\begin{aligned}L_0(4) &= \frac{(4-3)(4-5)}{(1-3)(1-5)} \\&= \frac{(1)(-1)}{(-2)(-4)} \\&= \frac{-1}{8} \\L_0(4) &= -0.125\end{aligned}$$

Paso 2: Calcular $L_1(x)$

$$\begin{aligned}L_1(4) &= \frac{(4-1)(4-5)}{(3-1)(3-5)} \\&= \frac{(3)(-1)}{(2)(-2)} \\&= \frac{-3}{-4} \\L_1(4) &= 0.75\end{aligned}$$

Paso 3: Calcular $L_2(x)$

$$\begin{aligned}L_2(4) &= \frac{(4-1)(4-3)}{(5-1)(5-3)} \\&= \frac{(3)(1)}{(4)(2)} \\&= \frac{3}{8} \\L_2(4) &= 0.375\end{aligned}$$

Paso 4: Sustituir en la fórmula

$$y = 1(-0.125) + 9(0.75) + 25(0.375)$$

$$y = -0.125 + 6.75 + 9.375$$

$$y = 16$$

Interpretación de resultados

La distancia aproximada recorrida a los 4 minutos es: **16 km**

Pseudocódigo

Inicio

Definir $x_0=1$, $y_0=1$

Definir $x_1=3$, $y_1=9$

Definir $x_2=5$, $y_2=25$

Definir $x=4$

$$L_0 = ((x-x_1)(x-x_2)) / ((x_0-x_1)(x_0-x_2))$$

$$L_1 = ((x-x_0)(x-x_2)) / ((x_1-x_0)(x_1-x_2))$$

$$L_2 = ((x-x_0)(x-x_1)) / ((x_2-x_0)(x_2-x_1))$$

$$y = (y_0 * L_0) + (y_1 * L_1) + (y_2 * L_2)$$

Mostrar y

Fin

Código en python

```
x0, y0 = 1, 1

x1, y1 = 3, 9

x2, y2 = 5, 25
```

```
x = 4

L0 = ((x-x1)*(x-x2)) / ((x0-x1)*(x0-x2))

L1 = ((x-x0)*(x-x2)) / ((x1-x0)*(x1-x2))

L2 = ((x-x0)*(x-x1)) / ((x2-x0)*(x2-x1))

y = y0*L0 + y1*L1 + y2*L2

print("Resultado:", y)
```

Resultado obtenido

Resultado: 16.0

Ejemplo 4

Planteamiento

Un ingeniero registra la altura de una pelota lanzada hacia arriba en distintos instantes de tiempo:

- A los 0 segundos la altura es de 1 metro.
- A los 1 segundos la altura es de 3 metros.
- A los 2 segundos la altura es de 2 metros.

Calcular la altura aproximada de la pelota a los 1.5 segundos utilizando interpolación cuadrática de Lagrange.

Fórmula y datos

Fórmula general

$$y = y_0 L_0(x) + y_1 L_1(x) + y_2 L_2(x)$$

Datos

$$x_0 = 0, \quad y_0 = 1$$

$$x_1 = 1, \quad y_1 = 3$$

$$x_2 = 2, \quad y_2 = 2$$

$$x = 1.5$$

Resolución

Paso 1: Calcular $L_0(x)$

$$L_0(1.5) = \frac{(1.5 - 1)(1.5 - 2)}{(0 - 1)(0 - 2)}$$

$$L_0(1.5) = \frac{-0.25}{2}$$

$$L_0(1.5) = -0.125$$

Paso 2: Calcular $L_1(x)$

$$L_1(1.5) = \frac{(1.5 - 0)(1.5 - 2)}{(1 - 0)(1 - 2)}$$

$$L_1(1.5) = \frac{-0.75}{-1}$$

$$L_1(1.5) = 0.75$$

Paso 3: Calcular $L_2(x)$

$$L_2(1.5) = \frac{(1.5 - 0)(1.5 - 1)}{(2 - 0)(2 - 1)}$$

$$L_2(1.5) = \frac{0.75}{2}$$

$$L_2(1.5) = 0.375$$

Paso 4: Sustituir en la fórmula general

$$y = y_0L_0(x) + y_1L_1(x) + y_2L_2(x)$$

Sustituyendo:

$$y = 1(-0.125) + 3(0.75) + 2(0.375)$$

Multiplicando:

$$y = -0.125 + 2.25 + 0.75$$

Sumando:

$$y = 2.875$$

Interpretación de resultados

La altura aproximada de la pelota a los 1.5 segundos es: **2.875 metros**

Pseudocódigo

Inicio

Definir $x_0=0$, $y_0=1$

Definir $x_1=1$, $y_1=3$

Definir $x_2=2$, $y_2=2$

Definir $x=1.5$

$$L_0 = ((x-x_1)(x-x_2))/((x_0-x_1)(x_0-x_2))$$

$$L_1 = ((x-x_0)(x-x_2))/((x_1-x_0)(x_1-x_2))$$

$$L_2 = ((x-x_0)(x-x_1))/((x_2-x_0)(x_2-x_1))$$

$$y = (y_0 * L_0) + (y_1 * L_1) + (y_2 * L_2)$$

Mostrar y

Fin

Código en python

```
# Datos

x0, y0 = 0, 1

x1, y1 = 1, 3

x2, y2 = 2, 2

# Valor a interpolar

x = 1.5

# Polinomios de Lagrange

L0 = ((x - x1)*(x - x2))/((x0 - x1)*(x0 - x2))

L1 = ((x - x0)*(x - x2))/((x1 - x0)*(x1 - x2))

L2 = ((x - x0)*(x - x1))/((x2 - x0)*(x2 - x1))

# Interpolación

y = y0*L0 + y1*L1 + y2*L2

# Resultado

print("Resultado:", y)
```

Resultado obtenido

Resultado: 2.875

INTERPOLACIÓN SEGMENTADA

Método numérico utilizado para estimar valores intermedios dentro de un conjunto de datos conocidos.

Cuota de Error del Método

En la interpolación segmentada (lineal), el error depende de la curvatura de la función real entre los puntos utilizados. Mientras más lineal sea el comportamiento de los datos, menor será el error.

Expresión general del error:

$$\bullet \quad E \propto (x - x_0)(x - x_1)$$

Donde:

- x = valor a interpolar
- x_0 y x_1 = puntos conocidos más cercanos

Ejemplo 1

Planteamiento

Un automóvil recorre una carretera y se registra la distancia recorrida en distintos tiempos.

Tiempo (h)	Distancia (km)
1	80
2	140
3	200
4	260

Calcular la distancia aproximada recorrida a las 2.5 horas usando interpolación segmentada.

Resolución

Como 2.5 está entre 2 y 3, se usa el segmento:

(2,140) y (3,200)

$$y = 140 + ((2.5 - 2)(200 - 140)) / (3 - 2)$$

$$y = 140 + ((0.5)(60))/1$$

$$y = 170$$

Resultado: 170 km

Seudocódigo

Inicio

Definir $x_0 = 2$

Definir $y_0 = 140$

Definir $x_1 = 3$

Definir $y_1 = 200$

Definir $x = 2.5$

$$y = y_0 + ((x - x_0)(y_1 - y_0)) / (x_1 - x_0)$$

Mostrar y

Fin

Código en Python

```
x0 = 2
y0 = 140
x1 = 3
y1 = 200
x = 2.5

y = y0 + ((x - x0) * (y1 - y0)) / (x1 - x0)

print("Distancia aproximada:", y, "km")
```

Distancia aproximada: 170.0 km

Ejemplo 2

Planteamiento

Una empresa registra las ventas durante el día.

Hora	Ventas (\$)
8	1200
10	1800
12	2500
14	3100

Calcular las ventas aproximadas a las 11 horas.

Resolución

11 está entre 10 y 12.

(10,1800) y (12,2500)

$$y = 1800 + ((11 - 10)(2500 - 1800)) / (12 - 10)$$

$$y = 2150$$

Resultado: 2150

Seudocódigo

Inicio

$$x0 = 10$$

$$y0 = 1800$$

$$x1 = 12$$

$$y1 = 2500$$

$$x = 11$$

$$y = y0 + ((x - x0)(y1 - y0)) / (x1 - x0)$$

Mostrar y

Fin

Código en Python

```
x0 = 10
y0 = 1800

x1 = 12
y1 = 2500

x = 11

y = y0 + ((x - x0) * (y1 - y0)) / (x1 - x0)

print("Ventas aproximadas:", y)

Ventas aproximadas: 2150.0
```

Ejemplo 3

Planteamiento

Se mide la altura de una planta durante varios días.

Día	Altura (cm)
2	15
4	25
6	40
8	52

Calcular la altura aproximada en el día 5.

Resolución

5 está entre 4 y 6.

(4,25) y (6,40)

$$y = 25 + ((5 - 4)(40 - 25)) / (6 - 4)$$

$$y = 32.5$$

Resultado: 32.5 cm

Seudocódigo

Inicio

$$x0 = 4$$

$$y0 = 25$$

$$x1 = 6$$

$$y1 = 40$$

$$x = 5$$

$$y = y0 + ((x - x0)*(y1 - y0))/(x1 - x0)$$

Mostrar y

Fin

Código en Python

```
x0 = 4

y0 = 25


x1 = 6

y1 = 40


x = 5


y = y0 + ((x - x0) * (y1 - y0)) / (x1 - x0)
```

```
print("Altura aproximada:", y, "cm")
```

```
Altura aproximada: 32.5 cm
```

Ejemplo 4

Planteamiento

La velocidad de una motocicleta cambia durante un recorrido.

Tiempo (s)	Velocidad (m/s)
0	0
5	20
10	35
15	50

Resolución

7 está entre 5 y 10.

(5,20) y (10,35)

$$y = 20 + ((7 - 5)(35 - 20)) / (10 - 5)$$

$$y = 26$$

Resultado: 26 m/s

Seudocódigo

Inicio

$$x0 = 5$$

$$y0 = 20$$

$$x1 = 10$$

$$y1 = 35$$

$$x = 7$$

$$y = y0 + ((x - x0)(y1 - y0)) / (x1 - x0)$$

Mostrar y

Fin

Código en Python

```
x0 = 5
y0 = 20

x1 = 10
y1 = 35

x = 7

y = y0 + ((x - x0) * (y1 - y0)) / (x1 - x0)

print("Velocidad aproximada:", y, "m/s")

-
Velocidad aproximada: 26.0 m/s
```

Correlación

Ejercicio 1:

Planteamiento:

Determinar el nivel de relación lineal (correlación) entre la cantidad de horas dedicadas al estudio y la calificación final obtenida en un examen de Métodos Numéricos por un grupo de 5 alumnos.

Datos:

Alumno	Horas de estudio (x)	Calificación (y)
1	2	50
2	4	70
3	6	85
4	8	90
5	10	95

Resolución:

- $n = 5$
- $\sum x = 30 \mid \sum y = 390$
- $\sum x^2 = 220 \mid \sum y^2 = 31850$
- $\sum xy = 2560$
- Fórmula de correlación de Pearson (r):

$$r = \frac{5(2560) - (30)(390)}{\sqrt{[5(220) - (30)^2][5(31850) - (390)^2]}}$$

- $r = \frac{12800 - 11700}{\sqrt{[1100 - 900][159250 - 152100]}} = \frac{1100}{\sqrt{(200)(7150)}} = \frac{1100}{1195.82} = 0.9198$

Interpretación de los Resultados:

Existe una correlación positiva muy fuerte ($r = 0.9198$). Esto significa que a medida que aumentan las horas de estudio, la calificación tiende a subir de manera proporcional.

Cuota de error:

El coeficiente de determinación es $r^2 = 0.846$. Esto indica que el modelo explica el 84.6% de los datos, dejando una cuota de error/varianza no explicada del 15.4%.

Pseudocódigo:

INICIO

Definir $n = 5$

Arrays horas = [2, 4, 6, 8, 10]

Arrays calif = [50, 70, 85, 90, 95]

sum_x = 0, sum_y = 0, sum_x2 = 0, sum_y2 = 0, sum_xy = 0

PARA $i = 0$ HASTA $n-1$ HACER:

 sum_x = sum_x + horas[i]

 sum_y = sum_y + calif[i]

 sum_x2 = sum_x2 + (horas[i] * horas[i])

 sum_y2 = sum_y2 + (calif[i] * calif[i])


```
sum_xy = sum_xy + (horas[i] * calif[i])  
FIN PARA
```

```
numerador = (n * sum_xy) - (sum_x * sum_y)  
denominador = RAIZ_CUADRADA((n * sum_x2 - sum_x^2) * (n * sum_y2 - sum_y^2))  
r = numerador / denominador  
error = (1 - (r * r)) * 100
```

```
IMPRIMIR "Correlación: ", r  
IMPRIMIR "Cuota de error (%): ", error  
FIN
```

Código:

```
import math
```

```
horas = [2, 4, 6, 8, 10]  
calif = [50, 70, 85, 90, 95]  
n = len(horas)
```

```
sum_x = sum(horas)  
sum_y = sum(calif)  
sum_x2 = sum(x**2 for x in horas)  
sum_y2 = sum(y**2 for y in calif)  
sum_xy = sum(horas[i] * calif[i] for i in range(n))
```

```
numerador = (n * sum_xy) - (sum_x * sum_y)  
denominador = math.sqrt((n * sum_x2 - sum_x**2) * (n * sum_y2 - sum_y**2))  
r = numerador / denominador
```

```
print(f"Coeficiente de correlación (r): {r:.4f}")  
print(f"Cuota de error no explicada: {(1 - r**2)*100:.2f}%")
```

Compilación:

```
Coeficiente de correlación (r): 0.9538  
Cuota de error no explicada: 9.02%
```

Ejercicio 2:

Planteamiento:

Evaluar si existe una relación estadística entre los años de antigüedad de una flotilla de vehículos y el costo anual de mantenimiento que generan (expresado en miles de pesos).

Datos:

Vehículo	Edad en años (x)	Costo (miles \$) (y)
1	1	1.5
2	3	2.0
3	5	3.5
4	7	4.0
5	9	5.5

Resolución:

- $n = 5$
- $\sum x = 25 \mid \sum y = 16.5$
- $\sum x^2 = 165 \mid \sum y^2 = 64.75$
- $\sum xy = 102.5$
- $$r = \frac{5(102.5) - (25)(16.5)}{\sqrt{[5(165) - (25)^2][5(64.75) - (16.5)^2]}}$$
- $$r = \frac{512.5 - 412.5}{\sqrt{[825 - 625][323.75 - 272.25]}} = \frac{100}{\sqrt{(200)(51.5)}} = \frac{100}{101.48} = 0.9853$$

Interpretación de los Resultados:

Se observa una correlación positiva casi perfecta ($r = 0.9853$). A mayor antigüedad del vehículo, el costo de mantenimiento sube de manera estrictamente lineal.

Cuota de error:

$r^2 = 0.970$. El error de varianza que escapa a este modelo lineal es mínimo, estableciendo una cuota de error de apenas el 3.0%.

Pseudocódigo:

INICIO

Definir $n = 5$

Arrays edad = [1, 3, 5, 7, 9]

Arrays costo = [1.5, 2.0, 3.5, 4.0, 5.5]

// ... (Inicialización de variables sumatorias en 0) ...

PARA $i = 0$ HASTA $n-1$ HACER:

$\text{sum_x} = \text{sum_x} + \text{edad}[i]$

$sum_y = sum_y + costo[i]$

$sum_x2 = sum_x2 + (edad[i]^2)$

$sum_y2 = sum_y2 + (costo[i]^2)$

$sum_xy = sum_xy + (edad[i] * costo[i])$

FIN PARA

$r = ((n * sum_xy) - (sum_x * sum_y)) / \text{RAIZ}(((n * sum_x2) - sum_x^2) * ((n * sum_y2) - sum_y^2))$

IMPRIMIR "Correlación (r): ", r

FIN

Código:

```
import math
```

```
edad = [1, 3, 5, 7, 9]
```

```
costo = [1.5, 2.0, 3.5, 4.0, 5.5]
```

```
n = len(edad)
```

```
# Funciones map/lambda para sumatorias rápidas
```

```
sum_x, sum_y = sum(edad), sum(costo)
```

```
sum_x2 = sum(map(lambda x: x**2, edad))
```

```
sum_y2 = sum(map(lambda x: x**2, costo))
```

```
sum_xy = sum(x*y for x, y in zip(edad, costo))
```

```
r = (n * sum_xy - sum_x * sum_y) / math.sqrt((n * sum_x2 - sum_x**2) * (n * sum_y2 - sum_y**2))
```

```
print(f"Correlación edad-costo (r): {r:.4f}")
```

```
print(f"Cuota de error del modelo: {(1 - r**2)*100:.1f}%")
```

Compilación:

```
Correlación edad-costo (r): 0.9853  
Cuota de error del modelo: 2.9%
```

Ejercicio 3:

Planteamiento:

Analizar mediante correlación de mínimos cuadrados cómo afecta la temperatura promedio mensual (°C) a la cantidad de abrigos vendidos en una sucursal, para prever inventarios.

Datos:

Mes	Temp °C (x)	Ventas (y)
1	5	80
2	10	65
3	15	40
4	20	25
5	25	10

Resolución:

- $n = 5$
- $\sum x = 75 \mid \sum y = 220$
- $\sum x^2 = 1375 \mid \sum y^2 = 12950$
- $\sum xy = 2400$
- $$r = \frac{5(2400) - (75)(220)}{\sqrt{[5(1375) - (75)^2][5(12950) - (220)^2]}}$$
- $$r = \frac{12000 - 16500}{\sqrt{[6875 - 5625][64750 - 48400]}} = \frac{-4500}{\sqrt{(1250)(16350)}} = -0.9944$$

Interpretación de los Resultados:

Existe una correlación negativa muy fuerte ($r = -0.9944$). Es una relación inversamente proporcional perfecta: a medida que sube la temperatura, las ventas caen drásticamente.

Cuota de error:

$r^2 = 0.988$. El modelo es sumamente preciso, con una cuota de error en la relación de los datos de apenas el 1.2%.

Pseudocódigo:

INICIO

Definir $n = 5$

Arrays temp = [5, 10, 15, 20, 25]

Arrays ventas = [80, 65, 40, 25, 10]

// Cálculo de sumatorias

PARA $i = 0$ HASTA $n-1$ HACER:

$\text{sum_x} = \text{sum_x} + \text{temp}[i]$

$\text{sum_y} = \text{sum_y} + \text{ventas}[i]$

$\text{sum_x2} = \text{sum_x2} + (\text{temp}[i] * \text{temp}[i])$

$\text{sum_y2} = \text{sum_y2} + (\text{ventas}[i] * \text{ventas}[i])$

$\text{sum_xy} = \text{sum_xy} + (\text{temp}[i] * \text{ventas}[i])$

FIN PARA

$\text{num} = (n * \text{sum_xy}) - (\text{sum_x} * \text{sum_y})$

$\text{den} = \text{RAIZ}((n * \text{sum_x2} - \text{sum_x} * \text{sum_x}) * (n * \text{sum_y2} - \text{sum_y} * \text{sum_y}))$

$r = \text{num} / \text{den}$

IMPRIMIR "Correlación Ventas-Clima: ", r

FIN

Código:

```
import math
```

```
temp = [5, 10, 15, 20, 25]
```

```
ventas = [80, 65, 40, 25, 10]
```

```
n = len(temp)
```

```
sum_x, sum_y = sum(temp), sum(ventas)
```

```
sum_x2 = sum(t**2 for t in temp)
```

```
sum_y2 = sum(v**2 for v in ventas)
```

```
sum_xy = sum(temp[i] * ventas[i] for i in range(n))
```

```
r = (n * sum_xy - sum_x * sum_y) / math.sqrt((n * sum_x2 - sum_x**2) * (n * sum_y2 - sum_y**2))
```

```
print(f"Coeficiente de correlación (r): {r:.4f} (Negativa)")
```

```
print(f"Cuota de error (varianza inexplicada): {(1 - r**2)*100:.2f}%")
```

Compilación:

```
Coeficiente de correlación (r): -0.9954 (Negativa)
Cuota de error (varianza inexplicada): 0.92%
```

Ejercicio 4:

Planteamiento:

Determinar si existe correlación entre la velocidad de subida contratada por un usuario (Mbps) y la latencia (Ping en ms) detectada al jugar en línea.

Datos:

Prueba	Velocidad Mbps (x)	Latencia ms (y)
1	10	120
2	20	90
3	30	75

4	50	60
5	100	45

Resolución:

- $n = 5$
- $\sum x = 210 \mid \sum y = 390$
- $\sum x^2 = 13500 \mid \sum y^2 = 33750$
- $\sum xy = 12750$
- $r = \frac{5(12750) - (210)(390)}{\sqrt{[5(13500) - (210)^2][5(33750) - (390)^2]}}$
- $r = \frac{63750 - 81900}{\sqrt{[67500 - 44100][168750 - 152100]}} = \frac{-18150}{\sqrt{(23400)(16650)}} = \frac{-18150}{19738.54} = -0.9195$

Interpretación de los Resultados:

Existe una correlación negativa fuerte ($r = -0.9195$). Demuestra que tener mayor ancho de banda tiende a reducir el ping en los juegos, aunque no es una relación lineal perfecta (debido a que llega un punto donde el ping no puede bajar de cero independientemente de la velocidad).

Cuota de error:

$r^2 = 0.845$. La cuota de error del ajuste lineal para este problema es del 15.5%, sugiriendo que la latencia también depende de factores externos a la velocidad.

Pseudocódigo:

INICIO

Definir $n = 5$

Arrays vel = [10, 20, 30, 50, 100]

Arrays ping = [120, 90, 75, 60, 45]

PARA i = 0 HASTA n-1 HACER:

sum_x = sum_x + vel[i]

sum_y = sum_y + ping[i]

sum_x2 = sum_x2 + (vel[i]^2)

sum_y2 = sum_y2 + (ping[i]^2)

sum_xy = sum_xy + (vel[i] * ping[i])

FIN PARA

correlacion = ((n * sum_xy) - (sum_x * sum_y)) / RAIZ(((n * sum_x2) - sum_x^2) * ((n * sum_y2) - sum_y^2))

cuota_error = 1 - (correlacion * correlacion)

IMPRIMIR "Correlación Velocidad-Ping: ", correlacion

IMPRIMIR "Cuota de error lineal: ", cuota_error

FIN

Código:

```
import math
```

```
vel = [10, 20, 30, 50, 100]
```

```
ping = [120, 90, 75, 60, 45]
```

```
n = len(vel)
```

```
sum_x, sum_y = sum(vel), sum(ping)
```

```
sum_x2 = sum(x**2 for x in vel)
```

```
sum_y2 = sum(y**2 for y in ping)
```

```
sum_xy = sum(vel[i] * ping[i] for i in range(n))
```

```
numerador = (n * sum_xy) - (sum_x * sum_y)
```

```
denominador = math.sqrt((n * sum_x2 - sum_x**2) * (n * sum_y2 - sum_y**2))
```

```
r = numerador / denominador
```

```
print(f"Coeficiente de correlación (r): {r:.4f}")
```

```
print(f"Cuota de error del modelo (%): {(1 - r**2)*100:.2f}%")
```

Compilació:

```
Coeficiente de correlación (r): -0.8826  
Cuota de error del modelo (%): 22.11%
```

Regresión

Ejercicio 1:

Planteamiento:

Una empresa de software desea generar un modelo predictivo para estimar las ganancias de su próxima aplicación (en miles de USD) basándose en el presupuesto invertido en campañas de publicidad (en miles de USD).

Datos:

Campana	Publicidad (x)	Ganancias (y)
1	2	20
2	3	25
3	5	34
4	6	38
5	8	50

Resolución:

- $n = 5$
- $\sum x = 24 \mid \sum y = 167$
- $\sum x^2 = 138 \mid \sum xy = 913$
- Cálculo de la pendiente (a_1):

$$a_1 = \frac{5(913) - (24)(167)}{5(138) - (24)^2} = \frac{4565 - 4008}{690 - 576} = \frac{557}{114} = 4.886$$

- Cálculo de la intersección (a_0):

$$a_0 = \bar{y} - a_1 \bar{x} = \frac{167}{5} - 4.886 \left(\frac{24}{5} \right) = 33.4 - 23.453 = 9.947$$

- Ecuación de regresión: $y = 9.947 + 4.886x$

Interpretación de los Resultados:

El modelo matemático obtenido es $y = 9.947 + 4.886x$. Esto indica que, sin inversión ($x=0$), las ganancias base son 9.94 miles, y por cada mil dólares extra en publicidad, las ganancias crecen 4.88 miles.

Cuota de error:

El Error Estándar de la Estimación ($SS_{\{y/x\}}$) es de 0.985 miles de USD. Este es el margen de error promedio si usamos la ecuación para predecir futuras ganancias.

Pseudocódigo:

INICIO

Definir $n = 5$

Arrays pub = [2, 3, 5, 6, 8]

Arrays gan = [20, 25, 34, 38, 50]

PARA $i = 0$ HASTA $n-1$ HACER:

```
sum_x = sum_x + pub[i]
```

```
sum_y = sum_y + gan[i]
```

```
sum_x2 = sum_x2 + (pub[i] * pub[i])
```

```
sum_xy = sum_xy + (pub[i] * gan[i])
```

```
FIN PARA
```

```
a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x^2)
```

```
a0 = (sum_y / n) - a1 * (sum_x / n)
```

```
// Cálculo de Error Estándar
```

```
suma_errores_cuadrado = 0
```

```
PARA i = 0 HASTA n-1 HACER:
```

```
    y_estimado = a0 + a1 * pub[i]
```

```
    suma_errores_cuadrado = suma_errores_cuadrado + (gan[i] - y_estimado)^2
```

```
FIN PARA
```

```
error_estandar = RAIZ(suma_errores_cuadrado / (n - 2))
```

```
IMPRIMIR "Recta: y = ", a0, " + ", a1, "x"
```

```
IMPRIMIR "Cuota Error (Syx): ", error_estandar
```

```
FIN
```

Código:

```
import math
```

```
x = [2, 3, 5, 6, 8]
```

```
y = [20, 25, 34, 38, 50]
```

```
n = len(x)
```

```
sum_x, sum_y = sum(x), sum(y)
```

```
sum_x2 = sum(i**2 for i in x)
```

```
sum_xy = sum(x[i] * y[i] for i in range(n))
```

```
a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x**2)
```

```
a0 = (sum_y / n) - a1 * (sum_x / n)
```

```
st_error_sum = sum((y[i] - (a0 + a1 * x[i]))**2 for i in range(n))
```

```
syx = math.sqrt(st_error_sum / (n - 2))
```

```
print(f"Modelo de Regresión: y = {a0:.3f} + {a1:.3f}x")
```

```
print(f"Cuota de Error (Syx): {syx:.3f}")
```

Compilación:

```
Modelo de Regresión: y = 9.947 + 4.886x
Cuota de Error (Syx): 0.984
```

Ejercicio 2:

Planteamiento:

Ajustar una curva lineal para predecir cuántos bugs (errores) se detectarán en la fase de testing de un módulo de software, basándose en la cantidad de líneas de código (medidas en cientos).

Datos:

Módulo	Líneas (Cientos) (x)	Bugs detectados (y)

A	1	2
B	2	4
C	4	7
D	6	11
E	8	14

Resolución:

- $n = 5$
- $\sum x = 21 \mid \sum y = 38$
- $\sum x^2 = 121 \mid \sum xy = 216$
- $a_1 = \frac{5(216) - (21)(38)}{5(121) - (21)^2} = \frac{1080 - 798}{605 - 441} = \frac{282}{164} = 1.719$
- $a_0 = \frac{38}{5} - 1.719 \left(\frac{21}{5} \right) = 7.6 - 7.219 = 0.381$
- **Ecuación de regresión: $y = 0.381 + 1.719x$**

Interpretación de los Resultados:

La ecuación $y = 0.381 + 1.719x$ modela el comportamiento del software. Nos dice que por cada 100 líneas de código agregadas ($x=1$), estadísticamente se introducirán ~1.72 bugs nuevos.

Cuota de error:

El error estándar calculado es de $S_{\{y/x\}} = 0.655$. La cuota de error del modelo predictivo es inferior a 1 bug, lo que lo hace sumamente confiable para auditorías de código.

Pseudocódigo:

INICIO

Definir $n = 5$

Arrays lineas = [1, 2, 4, 6, 8]

Arrays bugs = [2, 4, 7, 11, 14]

PARA $i = 0$ HASTA $n-1$ HACER:

// Acumuladores básicos

$sum_x = sum_x + lineas[i]$

$sum_y = sum_y + bugs[i]$

$sum_x2 = sum_x2 + (lineas[i] * lineas[i])$

$sum_xy = sum_xy + (lineas[i] * bugs[i])$

FIN PARA

$a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x * sum_x)$

$a0 = (sum_y / n) - a1 * (sum_x / n)$

IMPRIMIR "Ecuación: $y =$ ", $a0$, " $+$ ", $a1$, " x "

FIN

Código:

```
import math

lineas = [1, 2, 4, 6, 8]
bugs = [2, 4, 7, 11, 14]
n = len(lineas)

sum_x, sum_y = sum(lineas), sum(bugs)
sum_x2 = sum(x**2 for x in lineas)
sum_xy = sum(lineas[i] * bugs[i] for i in range(n))

a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x**2)
a0 = (sum_y / n) - a1 * (sum_x / n)

syx = math.sqrt(sum((bugs[i] - (a0 + a1 * lineas[i]))**2 for i in range(n)) / (n - 2))

print(f"Predicción de Bugs: y = {a0:.3f} + {a1:.3f}x")
print(f"Error Estándar (Syx): {syx:.3f} bugs")
```

Compilación:

```
Predicción de Bugs: y = 0.378 + 1.720x
Error Estándar (Syx): 0.271 bugs
```

Ejercicio 3:

Planteamiento:

Un administrador de base de datos necesita modelar cómo el número de nodos activos en un clúster (x) disminuye el tiempo de respuesta en minutos (y) de una consulta SQL de gran escala.

Datos:

Prueba	Nodos Activos (x)	Tiempo minutos (y)
1	2	50
2	4	35
3	6	25
4	8	15
5	10	10

Resolución:

- $n = 5$
- $\sum x = 30 \mid \sum y = 135$
- $\sum x^2 = 220 \mid \sum xy = 610$
- $a_1 = \frac{5(610) - (30)(135)}{5(220) - (30)^2} = \frac{3050 - 4050}{1100 - 900} = \frac{-1000}{200} = -5.0$
- $a_0 = \frac{135}{5} - (-5.0) \left(\frac{30}{5} \right) = 27 - (-25) = 52.0$
- Ecuación de regresión: $y = 52.0 - 5.0x$

Interpretación de los Resultados:

Obtenemos una regresión con pendiente negativa: $y = 52 - 5x$. Esto significa que por cada nodo extra que se enciende en el servidor, el tiempo de procesamiento disminuye de forma constante en 5 minutos.

Cuota de error:

El error de predicción del modelo ($S_{\{y/x\}}$) es de 7.18 minutos. Esta cuota indica que la reducción de tiempo no es perfectamente lineal (debido a los cuellos de botella de red), existiendo este margen de desviación en la estimación.

Pseudocódigo:

INICIO

Definir $n = 5$

Arrays nodos = [2, 4, 6, 8, 10]

Arrays tiempo = [50, 35, 25, 15, 10]

PARA $i = 0$ HASTA $n-1$ HACER:

$sum_x = sum_x + nodos[i]$

$sum_y = sum_y + tiempo[i]$

$sum_x2 = sum_x2 + (nodos[i] * nodos[i])$

$sum_xy = sum_xy + (nodos[i] * tiempo[i])$

FIN PARA

$a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x^2)$

$a0 = (sum_y / n) - a1 * (sum_x / n)$

IMPRIMIR "Modelo: Tiempo = ", $a0$, " ", $a1$, " * nodos"

FIN

Código:

```
import math
```

```
nodos = [2, 4, 6, 8, 10]
```

```
tiempo = [50, 35, 25, 15, 10]
```

```
n = len(nodos)
```

```
sum_x, sum_y = sum(nodos), sum(tiempo)
```

```
sum_x2 = sum(x**2 for x in nodos)
```

```
sum_xy = sum(nodos[i] * tiempo[i] for i in range(n))
```

```
a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x**2)
```

```
a0 = (sum_y / n) - a1 * (sum_x / n)
```

```
syx = math.sqrt(sum((tiempo[i] - (a0 + a1 * nodos[i]))**2 for i in range(n)) / (n - 2))
```

```
print(f"Ecuación: y = {a0:.1f} - {abs(a1):.1f}x")
```

```
print(f"Cuota de Error (Syx): {syx:.2f} min")
```

Compilació :

```
Ecuación: y = 57.0 - 5.0x  
Cuota de Error (Syx): 3.16 min
```

Ejercicio 4:

Planteamiento:

A partir de una encuesta local, se busca predecir mediante regresión lineal el salario mensual esperado (en miles de MXN) para un puesto de Ingeniería en Sistemas, basándose en los años de experiencia del candidato.

Datos:

Candidato	Experiencia Años (x)	Salario Miles MXN (y)
1	1	15
2	3	22
3	5	30
4	7	35
5	10	45

Resolución:

- $n = 5$
- $\sum x = 26 \mid \sum y = 147$
- $\sum x^2 = 184 \mid \sum xy = 926$
- $a_1 = \frac{5(926) - (26)(147)}{5(184) - (26)^2} = \frac{4630 - 3822}{920 - 676} = \frac{808}{244} = 3.311$
- $a_0 = \frac{147}{5} - 3.311 \left(\frac{26}{5} \right) = 29.4 - 3.311(5.2) = 12.183$
- Ecuación de regresión: $y = 12.183 + 3.311x$

Interpretación de los Resultados:

El modelo ($y = 12.183 + 3.311x$) refleja que el sueldo base (cero experiencia) arranca en aproximadamente \$12,183 MXN, y la tendencia dicta un aumento de \$3,311 MXN por cada año extra de experiencia.

Cuota de error:

El cálculo del error estándar da $S_{\{y/x\}} = 1.353$. El modelo predice los salarios con una cuota de error de tan solo \$1,353 MXN por encima o por debajo de la línea ideal, haciéndolo un ajuste excelente.

Pseudocódigo:

INICIO

Definir $n = 5$

Arrays exp = [1, 3, 5, 7, 10]

Arrays sueldo = [15, 22, 30, 35, 45]

// Realizar las 4 sumatorias (x, y, x2, xy) mediante un ciclo PARA

// ...

$$a1 = (n * \text{sum_xy} - \text{sum_x} * \text{sum_y}) / (n * \text{sum_x2} - \text{sum_x}^2)$$

$$a0 = (\text{sum_y} / n) - a1 * (\text{sum_x} / n)$$

// Calcular Error

SumaErr = 0

PARA i = 0 HASTA n-1 HACER:

 y_pred = a0 + a1 * exp[i]

 SumaErr = SumaErr + (sueldo[i] - y_pred)^2

FIN PARA

Syx = RAIZ(SumaErr / (n - 2))

IMPRIMIR "Sueldo esperado = ", a0, " + ", a1, " * años"

IMPRIMIR "Margen Error: ", Syx

FIN

Código:

```
import math
```

```
exp = [1, 3, 5, 7, 10]
```

```
sueldo = [15, 22, 30, 35, 45]
```

```
n = len(exp)
```

```
sum_x, sum_y = sum(exp), sum(sueldo)
```

```
sum_x2 = sum(x**2 for x in exp)
```

```
sum_xy = sum(exp[i] * sueldo[i] for i in range(n))
```

```
a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x**2)
```

```
a0 = (sum_y / n) - a1 * (sum_x / n)
```



```
syx = math.sqrt(sum((sueldo[i] - (a0 + a1 * exp[i]))**2 for i in range(n)) / (n - 2))
```

```
print(f"Modelo Predictivo Salarial: y = {a0:.3f} + {a1:.3f}x")
```

```
print(f"Cuota de Error (Syx): {syx:.3f} miles MXN")
```

Compilación:

```
Modelo Predictivo Salarial: y = 12.180 + 3.311x  
Cuota de Error (Syx): 0.830 miles MXN
```